# Core Concepts

## Resources

A resource is one YAML document with a `kind`, a unique `metadata.name`, `annotations` (key/value facts) and a `spec` (relations to other resources).

```yaml
apiVersion: architecture/v1alpha1
kind: ApplicationComponent
metadata:
  name: Archsight:Query:Engine
  annotations:
    architecture/description: Lexer, parser and evaluator of the query language.
    architecture/tags: query,parser
spec:
  realizedThrough:
    technologyArtifacts:
      - Repo:ionos-cloud:archsight
```

Names are unique per kind. Several documents can share one file, separated by `---`.

## Layers

| Layer | Kinds | Answers |
|---|---|---|
| Motivation | MotivationStakeholder, MotivationGoal, MotivationOutcome, MotivationRequirement, MotivationConstraint | Why? |
| Strategy | StrategyCapability | What ability do we need? |
| Business | BusinessActor, BusinessProduct, BusinessProcess, BusinessControl | Who does what? |
| Application | ApplicationService, ApplicationComponent, ApplicationInterface, DataObject | Which software? |
| Technology | TechnologyService, TechnologyArtifact, TechnologyInterface, TechnologySystemSoftware, TechnologyNode | How is it built and run? |
| Compliance | ComplianceEvidence | Can we prove it? |
| Tooling | View, Import, Analysis, Config | Queries, data sources, scripts |
| Docs | Page, PageMenu | The handbook you are reading |

Every kind has a reference page under `/doc/resources/<kind>` with its annotations, relations and a template.

## Relations

Relations are declared on one side and available on both. `realizedThrough` on a component shows up as `realizedBy` on the artifact. The verbs you will use most:

| Verb | Meaning |
|---|---|
| `realizedThrough` / `realizedBy` | Abstract thing implemented by a concrete one |
| `servedBy` / `serves` | Consumer uses a provider |
| `dependsOn` | Runtime or build dependency |
| `exposes` | Component offers an interface |
| `maintainedBy` / `contributedBy` | Ownership and participation of a team |

## Annotations

Annotations are namespaced keys such as `activity/status` or `architecture/description`. They are typed, searchable and can be shown as columns in a View. *Computed annotations* are derived from related resources when the database loads, for example the number of artifacts below a component. See Computed Annotations.

## The database

`Archsight::Database` loads all files, validates them, resolves relations in both directions and computes annotations. The UI **Reload** button, `archsight lint` and the MCP server all use this same loader, so what the linter accepts is what you will see.